

# Razor Pages en **ASP.NET** Core

Crea aplicaciones web sencillas con C#: agenda de teléfonos y lista de tareas

```
Index.cshtml.cs

public class IndexModel : PageModel
{
    public string Saludo { get; set; } = "¡Hola, alu

    public void OnGet() { }
}
```

# ¿Para quién es este curso?

Esta presentación está dirigida a estudiantes que ya conocen los fundamentos del lenguaje C# —variables, bucles, clases, métodos— y quieren dar el salto a desarrollar aplicaciones web completas usando el ecosistema .NET. No se requiere experiencia previa en desarrollo web, aunque conviene tener una idea básica de qué es HTML y una petición HTTP.

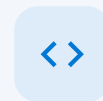
## REQUISITOS PREVIOS

- ✓ Conocer C# básico (tipos, clases, métodos)
- ✓ Saber qué es una clase y un objeto
- ✓ Tener instalado Visual Studio 2022 o VS Code
- ✓ Ganas de construir aplicaciones reales



**50**

diapositivas con teoría + práctica



**2**

proyectos completos paso a paso



**6h**

tiempo estimado de estudio

# Qué vamos a construir

Dos aplicaciones completas que cubren los patrones esenciales del desarrollo web con Razor Pages



## PROYECTO 1

### Agenda de teléfonos

Una aplicación CRUD clásica para gestionar contactos: nombre, teléfono, email. Aprenderás a crear páginas Razor, manejar formularios y validar datos — todo en memoria, sin base de datos.

CRUD

Formularios

Validación

En memoria



## PROYECTO 2

### Lista de tareas (To-Do)

Una aplicación de tareas pendientes con persistencia en base de datos SQLite usando Entity Framework Core. Marca tareas como completadas, añade nuevas y elimínalas.

EF Core

SQLite

Persistencia

Migraciones

## CONCEPTOS TRANSVERSALES

<> Razor syntax

🔗 Page Model

🧩 Tag Helpers

⚡ Async/await

# Índice del curso

Seis capítulos que te llevan desde cero hasta construir aplicaciones completas

01

## Fundamentos de la web en .NET

ASP.NET Core, Razor Pages, ventajas y arquitectura

---

02

## Tu primera aplicación

Crear el proyecto, estructura de carpetas, ejecución

---

03

## Sintaxis Razor

Expresiones, bloques de código, directivas, Tag Helpers

04

## El Page Model

PageModel, BindProperty, OnGet/OnPost, async

---

05

## Demo: Agenda de teléfonos

CRUD completo en memoria con validación

---

06

## Demo: Lista de tareas con BD

Entity Framework Core, SQLite, migraciones, CRUD

01 / 06

# Fundamentos de la web en .NET

Qué es ASP.NET Core, qué son las Razor Pages y por qué son ideales para empezar a desarrollar aplicaciones web con C#.



## CONCEPTO

# ¿Qué es ASP.NET Core?

ASP.NET Core es el framework web moderno, open source y multiplataforma de Microsoft. Permite construir aplicaciones web, APIs REST y microservicios con C#, y se ejecuta en Windows, Linux y macOS. Es la evolución del antiguo ASP.NET, reescrito desde cero para ser más rápido, modular y ligero.

Una aplicación ASP.NET Core se compone de un servidor web (Kestrel), un pipeline de middleware que procesa cada petición HTTP, y un sistema de inyección de dependencias integrado. Razor Pages es uno de los modelos de programación disponibles dentro de este framework.



### Alto rendimiento

Uno de los frameworks web más rápidos del mercado



### Multiplataforma

Windows, Linux y macOS



### Modular

NuGet packages, solo añades lo que usas



### Seguro

HTTPS, autenticación, CSRF integrado

# ¿Qué es Razor Pages?

Razor Pages es un modelo de programación dentro de ASP.NET Core que organiza la aplicación en **páginas** —no en controladores— donde cada página tiene su archivo `.cshtml` (vista HTML) y su archivo `.cshtml.cs` (lógica en C#). Es la forma más sencilla de construir aplicaciones web con C#.



## Basado en páginas

Cada URL del sitio corresponde a un archivo físico en la carpeta `Pages/`. La URL `/Contacto` se sirve desde `Pages/Contacto.cshtml`.



## Separación limpia

El HTML (`.cshtml`) y la lógica C# (`.cshtml.cs`) viven en archivos separados pero unidos por convención de nombres.



## Ideal para empezar

Curva de aprendizaje suave: no requiere entender el patrón MVC ni enrutamiento avanzado para construir tu primera app funcional.

# Razor Pages vs MVC vs Blazor

Tres formas de construir aplicaciones web en ASP.NET Core — cada una con sus puntos fuertes

| Criterio                       | Razor Pages <span>ELEGIDO</span> | MVC                   | Blazor            |
|--------------------------------|----------------------------------|-----------------------|-------------------|
| Organización                   | Por páginas                      | Por controladores     | Por componentes   |
| Curva de aprendizaje           | Baja                             | Media                 | Media-Alta        |
| Mejor para                     | Apps CRUD, formularios           | APIs y apps complejas | SPAs interactivas |
| Lenguaje en el cliente         | HTML + Razor                     | HTML + Razor          | C# (WebAssembly)  |
| Recomendado para principiantes | Sí                               | No tanto              | No al inicio      |

En este curso usamos **Razor Pages** porque es el modelo más natural para aplicaciones web con formularios y operaciones CRUD: agenda de teléfonos, lista de tareas, gestión de inventario, etc.



# Ventajas para principiantes

Seis razones por las que Razor Pages es la mejor puerta de entrada al desarrollo web con C#



01

## Sin enrutamiento complejo

La URL se deriva automáticamente de la ruta del archivo .cshtml dentro de Pages/. Cero configuración.



02

## Modelo por página

Cada página encapsula su lógica. No hay un controlador gigante con 30 acciones: el código está donde lo necesitas.



03

## Curva suave

Si sabes C#, en una hora estás construyendo páginas funcionales. No necesitas aprender JavaScript ni un framework frontend.



04

## Protección integrada

Las páginas POST incluyen automáticamente tokens anti-falsificación (CSRF). Validación de modelos incluida.



05

## Tag Helpers

Escribe HTML limpio con atributos especiales que generan formularios y enlaces tipados. Más legible que los HTML Helpers clásicos.



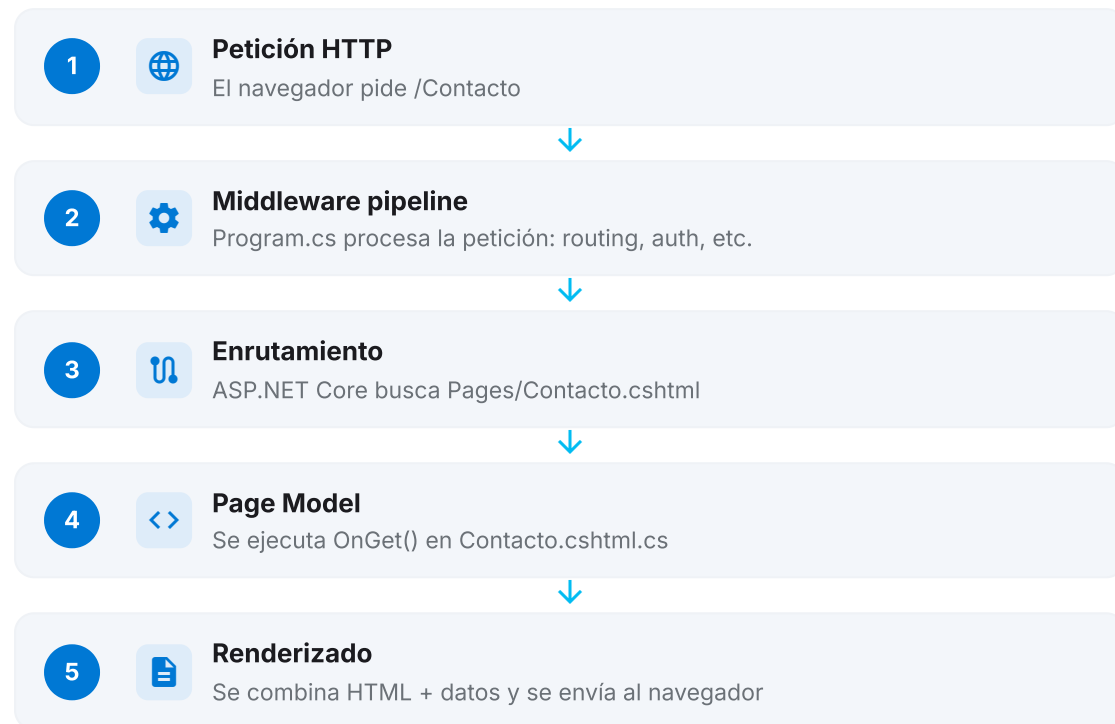
06

## Escalable

Empieza con una página simple y crece hasta aplicaciones profesionales. Razor Pages convive con APIs y Blazor en la misma app.

# Arquitectura de una aplicación Razor Pages

Cómo se procesa una petición HTTP desde el navegador hasta la página



Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddRazorPages();

var app = builder.Build();
app.UseStaticFiles();
app.UseRouting();
app.MapRazorPages();
app.Run();
```

Solo 4 líneas para arrancar una aplicación Razor Pages funcional.

# El patrón Page Model

En Razor Pages, cada página está formada por **dos archivos** que trabajan juntos por convención de nombres:

- 1 El archivo `.cshtml` contiene el HTML y las expresiones Razor. Define la estructura visual de la página.
- 2 El archivo `.cshtml.cs` contiene la clase PageModel, donde se escribe la lógica C#: cargar datos, procesar formularios, validar, etc.

El PageModel expone propiedades y métodos. Las propiedades se enlazan automáticamente al HTML mediante `@model`. Los métodos `OnGet`, `OnPost`, `OnPostEditar` responden a los verbos HTTP correspondientes.

Pages/Saludo.cshtml.cs

```
using Microsoft.AspNetCore.Mvc.RazorPages;

public class SaludoModel : PageModel
{
    // Propiedad disponible en la vista
    public string Mensaje { get; set; } = "";

    // Se ejecuta al hacer GET /Saludo
    public void OnGet()
    {
        Mensaje = "¡Bienvenido a Razor Pages!";
    }
}
```

El sufijo **Model** es convención. La vista referenciará esta clase con `@model SaludoModel`.

# Requisitos e instalación

Antes de empezar, instala estas herramientas. Todo es gratuito.



## .NET SDK 8.0 o superior

Descárgalo gratis desde [dotnet.microsoft.com](https://dotnet.microsoft.com). Incluye el runtime, el compilador y la CLI dotnet.



## Editor de código

Opciones recomendadas: Visual Studio 2022 Community (Windows/Mac) o Visual Studio Code con la extensión C# Dev Kit (multiplataforma).



## SQLite (opcional)

Para el capítulo 6 (lista de tareas). EF Core usa SQLite por defecto, no requiere instalación separada.



## Terminal / línea de comandos

Cualquier terminal sirve: PowerShell, CMD, bash o la integrada en VS Code (Ctrl+ñ).

bash — verificación

```
# Comprobar la versión instalada
dotnet --version

# Debería mostrar 8.0.x o superior

# Listar los SDK instalados
dotnet --list-sdks

# Crear tu primera app Razor Pages
dotnet new razor -o MiAgenda
cd MiAgenda
dotnet run
```

**i** La primera ejecución descarga paquetes NuGet. Puede tardar 30-60 segundos.

# 02 / 06

---

## Tu primera aplicación

Crea tu primer proyecto Razor Pages, entiende su estructura de carpetas y ejecuta la aplicación en el navegador.



## PASO 1

# Crear el proyecto

Tres comandos en la terminal y tendrás tu primera aplicación web funcionando

### 1 Crear el proyecto

Abre una terminal en la carpeta donde quieras crear el proyecto y ejecuta:

```
dotnet new razor -o MiAgenda
```

Esto crea una carpeta MiAgenda con la plantilla Razor Pages lista para ejecutar.

### 2 Entrar en la carpeta

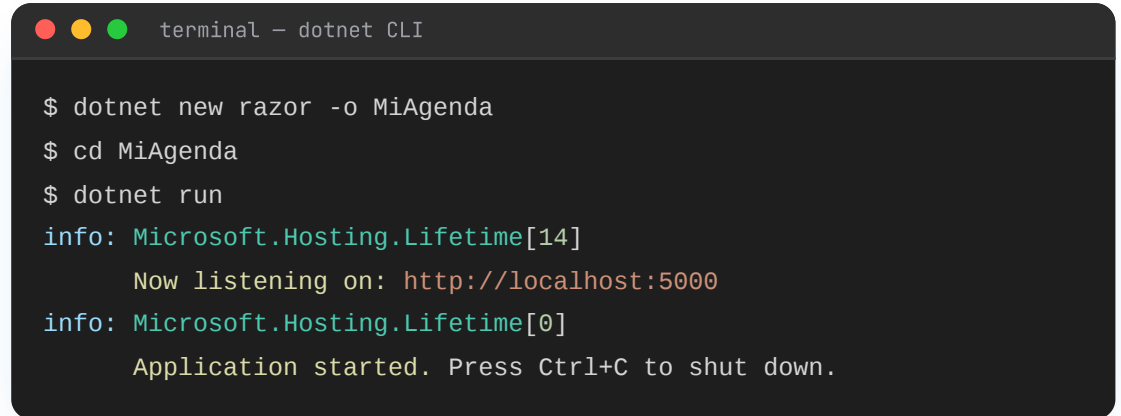
```
cd MiAgenda
```

Todos los comandos siguientes se ejecutan desde dentro del proyecto.

### 3 Ejecutar la aplicación

```
dotnet run
```

La primera vez tarda un poco mientras restaura paquetes. Verás algo como: Now listening on: http://localhost:5000



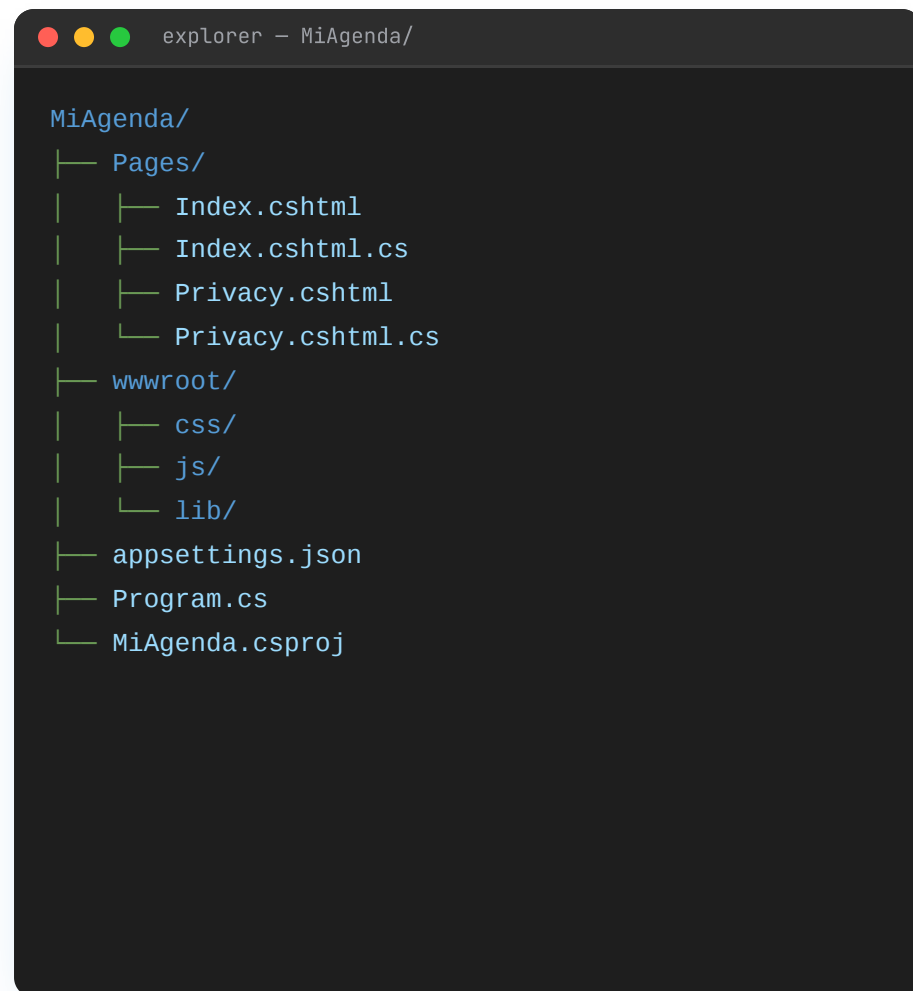
```
terminal - dotnet CLI

$ dotnet new razor -o MiAgenda
$ cd MiAgenda
$ dotnet run
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
```

Abre <http://localhost:5000> en tu navegador y verás la plantilla por defecto de Razor Pages.

# Estructura de carpetas

Cada archivo tiene un papel. Conocerla te ahorrará tiempo y confusiones.



## Pages/ — Las páginas

Carpeta donde viven TODAS las páginas `.cshtml` y `.cshtml.cs`. La ruta del archivo = la URL.  
`Pages/Contacto.cshtml` → `/Contacto`.



## wwwroot/ — Archivos estáticos

CSS, JavaScript, imágenes, fuentes. Sirven directamente sin pasar por C#. Aquí pondrás tus estilos personalizados.



## Program.cs — Configuración

Punto de entrada de la app. Define servicios, middleware y enrutamiento. En .NET 6+ reemplaza al antiguo `Startup.cs`.



## appsettings.json + .csproj

`appsettings.json` : cadenas de conexión, claves, configuración. El `.csproj` define el SDK, paquetes NuGet y versión de .NET.

# Program.cs: el punto de entrada

```
Program.cs

var builder = WebApplication.CreateBuilder(args);

// Registrar servicios
builder.Services.AddRazorPages();

var app = builder.Build();

// Pipeline de middleware
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
else
{
    app.UseExceptionHandler("/Error");
}

app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

// Mapear páginas Razor a URLs
app.MapRazorPages();
```



## AddRazorPages()

Registra los servicios internos que Razor Pages necesita (model binding, validación, etc.).



## Exception handling

En desarrollo muestra errores detallados. En producción redirige a /Error para no filtrar información sensible.



## UseStaticFiles()

Activa la carpeta wwwroot/ para servir CSS, JS e imágenes directamente al navegador.



## MapRazorPages()

Conecta automáticamente URLs con archivos en Pages/. La convención lo hace todo.



## \_ViewImports y \_ViewStart

Dos archivos con nombre reservado que configuran todas las páginas de la carpeta Pages/

### \_ViewImports.cshtml

```
_ViewImports.cshtml

@using MiAgenda
@using MiAgenda.Data
@namespace MiAgenda.Pages
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Define directivas que se aplican a **TODAS** las páginas de la carpeta (y subcarpetas). Aquí importas namespaces de C#, registras Tag Helpers y estableces el namespace por defecto. Si añades una carpeta `Pages/Admin/`, puedes crear su propio `_ViewImports` para añadir using específicos de administración.

### \_ViewStart.cshtml

```
_ViewStart.cshtml

@{
    Layout = "_Layout";
}
```

Se ejecuta **ANTES** que cualquier página. Lo más común es definir aquí el layout común ( `_Layout.cshtml` ) para que todas las páginas compartan la misma estructura HTML (cabecera, navegación, pie de página). Si una página específica no quiere layout, puede sobrescribirlo con `Layout = null;` en su propio código.

# El layout compartido (`_Layout.cshtml`)

Una sola plantilla HTML envuelve a todas tus páginas — evita repetir cabeceras y pies

PAGES/SHARED/\_LAYOUT.CSHTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <title>@ViewData["Title"] - Mi Agenda</title>
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <header>
    <nav>
      <a href="/">Inicio</a>
      <a href="/Contactos">Contactos</a>
      <a href="/Tareas">Tareas</a>
    </nav>
  </header>

  <main>
    @RenderBody()
  </main>

  <footer>&copy; 2026 - Mi Agenda</footer>
</body>
</html>
```

## → @RenderBody()

Marcador donde se inserta el contenido de cada página. Es lo único obligatorio en un layout.

## T @ViewData["Title"]

Cada página puede establecer su título así: `@{ ViewData["Title"] = "Contactos"; }` en la primera línea del .cshtml.

## ↻ Reutilización

Cualquier cambio en `_Layout.cshtml` se refleja en **TODAS** las páginas. Cambia el menú una vez y se actualiza en todas partes.

# Navegación entre páginas

Dos formas de crear enlaces: HTML tradicional o Tag Helper tipado

HTML TRADICIONAL — sencillo

```
<a href="/Contactos">Ver contactos</a>
<a href="/Contactos/Crear">Añadir contacto</a>
```

&lt;&gt;

TAG HELPER ASP-PAGE — recomendado

⚙️

```
<a asp-page="/Contactos" asp-route-id="5">
  Ver contacto 5
</a>

<form method="post" asp-page="/Contactos"
  asp-page-handler="Eliminar">
  <input type="hidden" name="id" value="5" />
  <button type="submit">Borrar</button>
</form>
```



## Tipado seguro

`asp-page` verifica que la página existe al compilar. Si renombas `Contactos.cshtml`, el compilador avisa de los enlaces rotos.



## asp-route-\*

Genera automáticamente la URL con parámetros: `/Contactos/Editar/5` a partir de `asp-route-id="5"`.



## asp-page-handler

Apunta a un método específico del Page Model: `asp-page-handler="Eliminar"` ejecuta `OnPostEliminar()`.

## 🚩 PASO 5

# Ejecutar la aplicación

## COMANDOS CLI

```
# Ejecutar en modo desarrollo (recarga cambios)
dotnet run

# O con watch (hot reload – recarga al guardar)
dotnet watch run

# Compilar sin ejecutar (verificar errores)
dotnet build

# Publicar para producción
dotnet publish -c Release
```

## SALIDA TÍPICA

```
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5000
```



## PROBLEMAS COMUNES Y SOLUCIONES



### Puerto en uso

Si el puerto 5000 está ocupado, ASP.NET Core usa automáticamente otro (5001, 5002). Mira el mensaje en la terminal.



### Cambios no se reflejan

Usa `dotnet watch run` para hot reload. Si no, detén con `Ctrl+C` y vuelve a ejecutar `dotnet run`.



### Pantalla de error amarilla

En desarrollo, ASP.NET muestra errores detallados. Lee la primera línea del stack trace: suele indicar el archivo y la línea exactas.

# 03 / 06

## Sintaxis Razor

Aprende a combinar HTML y C# en el mismo archivo con la sintaxis @, expresiones en línea, bloques de código y directivas.

03



## 💡 CONCEPTO

# ¿Qué es Razor? La sintaxis @

Razor es el motor de plantillas que mezcla HTML y C# usando el símbolo @

Razor te permite escribir código C# directamente dentro del HTML. El símbolo @ marca la transición de HTML a C#: cuando Razor ve una @, interpreta lo que sigue como código C# y lo evalúa.

Es más compacto y legible que otras alternativas como PHP o los Web Forms clásicos. No reemplaza a HTML: lo extiende. El resultado final siempre es HTML que el navegador puede procesar.



## REGLA DE ORO

Una @ simple inserta el valor de una expresión. Un bloque @{ ... } ejecuta varias sentencias sin imprimir nada.

## EJEMPLOS.CSHTML



```
<!-- Expresión simple -->
<h1>Hola, @nombre!</h1>

<!-- Propiedad del modelo -->
<p>Tienes @Model.Contactos.Count contactos.</p>

<!-- Llamada a método -->
<p>Fecha: @DateTime.Now.ToString("dd/MM/yyyy")</p>

<!-- Operación -->
<p>Total: @(precio * cantidad) €</p>
```

**i** Los paréntesis en @(precio \* cantidad) delimitan la expresión completa. Sin ellos, Razor solo evaluaría @precio y dejaría \* cantidad como texto.

# Expresiones en línea y bloques de código

## { } Expresiones en línea — una sola @

```
@* Comentario Razor — no sale en el HTML final *@

<p>Usuario: @User.Identity.Name</p>

<p>Precio: @producto.Precio.ToString("C")</p>

<p>Con IVA: @(precio * 1.21) €</p>

@if (esMayorEdad) {
    <span class="badge">Adulto</span>
}
```

Una sola @ evalúa **UNA** expresión y la imprime en el HTML. Úsalas para valores cortos: propiedades, llamadas a métodos, operaciones aritméticas. Si la expresión tiene operadores, envuélvela en paréntesis.

## [] Bloques de código — @{ ... }

```
@{
    var nombre = "Ana";
    var edad = 25;
    var mensaje = $"{nombre} tiene {edad} años";
    var productos = new List<string> { "Manzana", "Pera" };
}

<p>@mensaje</p>

<p>Total productos: @productos.Count</p>
```

El bloque @{ ... } ejecuta varias líneas de C# **SIN** imprimir nada directamente. Declara variables, hace cálculos, prepara datos. Las variables declaradas ahí quedan disponibles en el resto de la página con @nombreVariable .

# Variables y tipos en Razor

Puedes declarar variables C# con `var` o con el tipo explícito — igual que en cualquier archivo `.cs`

DECLARACION.CSHTML

<> TIPOS COMUNES



```
@{
    // Tipos básicos
    var nombre = "Carlos";           // string
    int edad = 30;                   // int
    decimal precio = 19.95m;         // decimal (¡con la m!)
    bool activo = true;              // bool
    DateTime hoy = DateTime.Today;   // DateTime

    // Colecciones
    var colores = new List<string>
    {
        "Rojo", "Verde", "Azul"
    };

    // String interpolation
    var saludo = $"Hola, {nombre}. " +
                $"Tienes {edad} años";
}

<p>@saludo</p>
<p>Color favorito: @colores[0]</p>
```

| Tipo C#  | Uso             | Ejemplo         |
|----------|-----------------|-----------------|
| string   | Texto           | "Hola"          |
| int      | Enteros         | 42              |
| decimal  | Dinero          | 19.95m          |
| bool     | Verdadero/falso | true            |
| DateTime | Fechas          | DateTime.Now    |
| List<T>  | Listas          | new List<int>() |
| var      | Inferencia      | var x = 5       |



## Estructuras de control: @if, @switch

### 🔗 @if / else if / else

```
@if (Model.Contactos.Count == 0)
{
    <div class="alert alert-info">
        No tienes contactos todavía.
        <a asp-page="/Crear">Añadir el primero</a>
    </div>
}
else if (Model.Contactos.Count < 5)
{
    <p>Tienes pocos contactos (@Model.Contactos.Count).</p>
}
else
{
    <p>Tienes @Model.Contactos.Count contactos.</p>
}
```

### 🚦 @switch

```
@switch (Model.Rol)
{
    case "admin":
        <span class="badge badge-danger">Admin</span>
        break;
    case "editor":
        <span class="badge badge-warning">Editor</span>
        break;
    case "usuario":
        <span class="badge badge-info">Usuario</span>
        break;
    default:
        <span class="badge">Invitado</span>
        break;
}
```

💡 Las llaves { } marcan el bloque de HTML que se renderiza cuando se cumple la condición. Razor detecta automáticamente qué es C# y qué es HTML según el contexto.

# Bucles: @for, @foreach, @while

Tres formas de repetir HTML para cada elemento de una colección

## @foreach — el más usado

```
<ul>
@foreach (var contacto in Model.Contactos)
{
    <li>
        <strong>@contacto.Nombre</strong>
        - @contacto.Telefono
    </li>
}
</ul>
```

El bucle más común en Razor. Recorre cualquier `IEnumerable<T>` y repite el HTML interior para cada elemento.

## @for y @while

```
@* @for: cuando necesitas el índice *@
<table>
@for (int i = 0; i < Model.Contactos.Count; i++)
{
    <tr>
        <td>@(i + 1)</td>
        <td>@Model.Contactos[i].Nombre</td>
    </tr>
}
</table>

@* @while: raro en vistas, pero existe *@
@{ int n = 0; }
@while (n < 5)
{
    <p>Línea @n</p>
    n++;
}
```



Para iterar con índice y necesitas saber si es el último elemento, usa `@for`. Para simplemente recorrer una lista, `@foreach` es siempre mejor.

# Directivas: @page, @model, @using

Las directivas controlan cómo Razor procesa la página

CONTACTOS/INDEX.CSHTML

```
@page
@model MiAgenda.Pages.Contactos.IndexModel
@using MiAgenda.Models
@{
    ViewData["Title"] = "Mis contactos";
}

<h1>@ViewData["Title"]</h1>

@foreach (var c in Model.Contactos)
{
    <p>@c.Nombre - @c.Telefono</p>
}
```

| Directiva             | Qué hace                                                                                     |
|-----------------------|----------------------------------------------------------------------------------------------|
| @page                 | Convierte el archivo en una página Razor (no una vista parcial). Debe ir siempre la primera. |
| @page "{id:int?}"     | Define un parámetro de ruta opcional.<br>/Contactos/Editar/5                                 |
| @model Tipo           | Declara el tipo del Page Model. Accesible vía @Model en la vista.                            |
| @using Namespace      | Importa un namespace C#, igual que en .cs.                                                   |
| @inject Servicio Tipo | Inyecta un servicio del contenedor DI directamente en la página.                             |

! @page DEBE ser la primera línea del archivo. Sin ella, Razor trata el archivo como una vista parcial y no se sirve desde una URL.

# Tag Helpers

Atributos HTML especiales que generan formularios y enlaces tipados — más limpios que los HTML Helpers

Los Tag Helpers son atributos que empiezan por `asp-` y se añaden a etiquetas HTML normales. Al renderizar, ASP.NET Core los reemplaza por HTML final. La ventaja: escribes HTML que se parece al resultado final, no una llamada a método C#.

Para activarlos en todo el proyecto, la línea `@addTagHelper *`, `Microsoft.AspNetCore.Mvc.TagHelpers` debe estar en `_ViewImports.cshtml`. La plantilla por defecto ya la incluye.

## LOS MÁS USADOS

- `asp-for` — vincula un campo de formulario a una propiedad del modelo
- `asp-page` — genera enlace a otra página Razor
- `asp-route-*` — añade parámetros a la URL
- `asp-page-handler` — apunta a un método concreto del PageModel
- `asp-validation-for` — muestra el mensaje de validación de un campo

HTML HELPER — clásico, evítalo



```
@Html.ActionLink("Editar", "Editar",  
    new { id = 5 })
```

TAG HELPER — moderno, recomendado



```
<a asp-page="/Contactos/Editar"  
    asp-route-id="5">Editar</a>  
  
<form method="post">  
    <input asp-for="Contacto.Nombre" />  
    <span asp-validation-for="Contacto.Nombre">  
    </span>  
    <button type="submit">Guardar</button>  
</form>
```

# 04 / 06

## El Page Model

Donde vive la lógica C# de cada página: cargar datos, procesar formularios, validar y redirigir.

# 04



## 💡 CONCEPTO

# La clase `PageModel`

La clase base de la que heredan todos los modelos de página

CONTACTOS/INDEX.CSHTML.CS

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace MiAgenda.Pages.Contactos
{
    public class IndexModel : PageModel
    {
        private readonly AgendaService _servicio;

        // Inyección por constructor
        public IndexModel(AgendaService servicio)
        {
            _servicio = servicio;
        }

        // Propiedad expuesta a la vista
        public List<Contacto> Contactos { get; set; }

        // Manejador GET
        public void OnGet()
        {
            Contactos = _servicio.ObtenerTodos();
        }
    }
}
```

<> 4 PUNTOS CLAVE



### Hereda de `PageModel`

Es obligatorio. Trae toda la maquinaria: `ModelState`, `TempData`, `Redirect`, etc.



### Inyección de dependencias

Los servicios se inyectan por constructor. ASP.NET Core los resuelve automáticamente.



### Propiedades públicas

Solo las propiedades públicas son accesibles desde la vista vía `@Model`. Las privadas no.



### Métodos manejadores

`OnGet`, `OnPost`, `OnGetEditor` ... Cada uno responde a un verbo HTTP + handler concreto.

# Enlace de datos: [BindProperty]

Cómo los datos del formulario llegan automáticamente a tus propiedades C#

CONTACTOS/CREAR.CSHTML.CS

```
public class CrearModel : PageModel
{
    // Se enlaza automáticamente al recibir POST
    [BindProperty]
    public Contacto Contacto { get; set; }

    public void OnGet()
    {
        // Formulario vacío, mostrar página
    }

    public IActionResult OnPost()
    {
        if (!ModelState.IsValid)
        {
            return Page(); // Vuelve a mostrar el form
        }

        // Contacto ya está relleno con los datos del form
        _servicio.Añadir(Contacto);
        return RedirectToPage("Index");
    }
}
```

<> CÓMO FUNCIONA



## [BindProperty]

Recoge automáticamente los campos del formulario HTTP cuyos nombres coinciden con las propiedades del objeto. Si el form tiene `name="Contacto.Nombre"`, rellena `Contacto.Nombre`.



## Solo en POST por defecto

Por defecto, `[BindProperty]` solo enlaza en peticiones POST. Si necesitas enlazar en GET, usa `[BindProperty(SupportsGet = true)]`.



## ModelState.IsValid

Verifica las anotaciones de validación `[Required]`, `[StringLength]`, etc. Si falla, `IsValid` es false y vuelves a mostrar el form con errores.

# Métodos `OnGet` y `OnPost`

Dos métodos para dos verbos HTTP: GET para mostrar, POST para procesar

## 👁 OnGet — mostrar página

```
public class EditarModel : PageModel
{
    [BindProperty]
    public Contacto Contacto { get; set; }

    // GET /Contactos/Editar/5
    public IActionResult OnGet(int id)
    {
        Contacto = _servicio.ObtenerPorId(id);

        if (Contacto == null)
        {
            return NotFound();
        }

        return Page();
    }
}
```

**OnGet** se ejecuta cuando el navegador pide la URL. Recibe parámetros de la ruta ( `id` ) o de la query string ( `?nombre=...` ). Suele cargar datos para mostrarlos en la página.

## ➤ OnPost — procesar formulario

```
// POST /Contactos/Editar/5
public IActionResult OnPost()
{
    if (!ModelState.IsValid)
    {
        return Page();
    }

    _servicio.Actualizar(Contacto);

    // Patrón PRG: Post-Redirect-Get
    return RedirectToPage("Index");
}
```

**OnPost** se ejecuta al enviar un formulario `method="post"` . Aquí validas, guardas y **SIEMPRE** rediriges. Redirigir evita que el formulario se reenvíe al recargar la página (patrón PRG).



# OnPostAsync con async/await

Operaciones de base de datos o APIs deben ser async para no bloquear el servidor

CREAR.CSHTML.CS

```
public class CrearModel : PageModel
{
    private readonly AgendaDbContext _db;

    public CrearModel(AgendaDbContext db)
    {
        _db = db;
    }

    [BindProperty]
    public Contacto Contacto { get; set; }

    public async Task<IActionResult> OnPostAsync()
    {
        if (!ModelState.IsValid)
        {
            return Page();
        }

        // await libera el hilo mientras espera la BD
        _db.Contactos.Add(Contacto);
        await _db.SaveChangesAsync();

        return RedirectToPage("Index");
    }
}
```

<> BUENAS PRÁCTICAS ASYNC



## Task<IActionResult>

Versión async del método. El sufijo `Async` es convención. Devuelve `Task<IActionResult>` en vez de `IActionResult`.



## await no bloquea

Mientras la BD responde, el hilo se libera para atender otras peticiones. Mejora el rendimiento **x10** con muchos usuarios concurrentes.



## Solo para I/O

Usa async solo en operaciones de I/O: BD, ficheros, HTTP. Para cálculos CPU no aporta nada y complica el código.

# Manejo de formularios

Patrón completo: GET muestra el form vacío, POST recibe los datos y los procesa

CREAR.CSHTML

```
@page
@model CrearModel

<form method="post" class="form">
    @* Token anti-CSRF automático *@
    <div>
        <label asp-for="Contacto.Nombre"></label>
        <input asp-for="Contacto.Nombre" />
        <span asp-validation-for="Contacto.Nombre">
        </span>
    </div>

    <div>
        <label asp-for="Contacto.Telefono"></label>
        <input asp-for="Contacto.Telefono" />
        <span asp-validation-for="Contacto.Telefono">
        </span>
    </div>

    <button type="submit">Guardar</button>
</form>
```

<> CÓMO FUNCIONA



## Anti-CSRF automático

Razor Pages añade automáticamente un token oculto que protege contra ataques CSRF. No tienes que hacer nada.



## asp-for="..."

Genera el `<label>` con el texto correcto (desde `[Display]`) y el atributo `for` con el id del input. Infiere el tipo: si la propiedad es `int`, usa `input type="number"`.



## asp-validation-for="..."

Muestra el mensaje de error si la validación falla. Lee de `ModelState`, que se rellena con las anotaciones del modelo.



# Redirecciones y TempData

Cómo pasar datos entre páginas cuando rediriges

```
// Pages/Contactos/Crear.cshtml.cs
public class CrearModel : PageModel
{
    public IActionResult OnPost()
    {
        if (!ModelState.IsValid) return Page();

        _servicio.Añadir(Contacto);

        // Mensaje que sobrevive a la redirección
        TempData["Mensaje"] = "Contacto añadido correctamente";

        return RedirectToPage("Index");
    }
}

// En Index.cshtml.cs
public void OnGet()
{
    Mensaje = TempData["Mensaje"] as string;
}
```

## RedirectToPage("Index")

Redirige a otra página Razor por nombre. La URL cambia en el navegador. Recarga limpia: si el usuario pulsa F5 no se reenvía el POST.



## TempData

Diccionario que guarda datos durante UNA petición extra. Útil para mensajes de confirmación tras redirigir. Se borra automáticamente al leerlo.



## TempData.Keep()

Si quieres que TempData sobreviva más redirecciones, llama a TempData.Keep("clave") tras leerlo.



**Patrón PRG:** Post → Redirect → Get evita reenvíos duplicados al recargar.

# 05 / 06

---

## Agenda de teléfonos

Tu primera aplicación CRUD completa: listar, crear, editar y borrar contactos — en memoria, sin base de datos.

 Listar    Crear    Editar    Borrar



# Modelo de datos: Contacto

Una clase C# simple representa un contacto de la agenda

```
// Models/Contacto.cs
using System.ComponentModel.DataAnnotations;

namespace MiAgenda.Models
{
    public class Contacto
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "El nombre es obligatorio")]
        [StringLength(50,
            ErrorMessage = "Máximo 50 caracteres")]
        [Display(Name = "Nombre completo")]
        public string Nombre { get; set; }

        [Required(ErrorMessage = "El teléfono es obligatorio")]
        [Phone(ErrorMessage = "Teléfono no válido")]
        [Display(Name = "Teléfono")]
        public string Telefono { get; set; }

        [EmailAddress(ErrorMessage = "Email no válido")]
        [Display(Name = "Correo electrónico")]
        public string Email { get; set; }
    }
}
```

| ATRIBUTO          | QUÉ VALIDA                      |
|-------------------|---------------------------------|
| [Required]        | Campo obligatorio               |
| [StringLength(n)] | Longitud máxima                 |
| [Phone]           | Formato de teléfono             |
| [EmailAddress]    | Formato de email                |
| [Display(Name=)]  | Etiqueta visible en formularios |
| [Range(min,max)]  | Número entre dos valores        |



Las anotaciones de `System.ComponentModel.DataAnnotations` validan automáticamente en servidor. Si el cliente tiene JavaScript activo, ASP.NET genera validación en cliente también.



# Almacenamiento en memoria

Un servicio singleton guarda los contactos mientras la app está en marcha

```
// Services/AgendaService.cs
using MiAgenda.Models;

namespace MiAgenda.Services
{
    public class AgendaService
    {
        private readonly List<Contacto> _contactos = new();
        private int _nextId = 1;

        public List<Contacto> ObtenerTodos()
        {
            return _contactos
                .OrderBy(c => c.Nombre)
                .ToList();
        }

        public Contacto? ObtenerPorId(int id)
        {
            return _contactos
                .FirstOrDefault(c => c.Id == id);
        }

        public void Añadir(Contacto c)
        {
            c.Id = _nextId++;
            _contactos.Add(c);
        }

        public void Actualizar(Contacto c)
```

## Por qué en memoria

Para esta demo usamos una lista en memoria. Al reiniciar la app, los datos se pierden. En el capítulo 6 veremos cómo persistirlos en SQLite con Entity Framework Core.

```
// Program.cs
builder.Services.AddSingleton<AgendaService>();
```

## AddSingleton

Crea **UNA instancia** compartida por toda la app. Todos los usuarios ven los mismos datos — perfecto para una demo de aula.

CICLO DE VIDA

App entera

ESTADO

Compartido

PERSISTENCIA

No



## Página Index: listar contactos

```
// Pages/Contactos/Index.cshtml.cs
using Microsoft.AspNetCore.Mvc.RazorPages;
using MiAgenda.Models;
using MiAgenda.Services;

namespace MiAgenda.Pages.Contactos
{
    public class IndexModel : PageModel
    {
        private readonly AgendaService _svc;

        public IndexModel(AgendaService svc)
        {
            _svc = svc;
        }

        public List<Contacto> Contactos { get; set; }
        public string? Mensaje { get; set; }

        public void OnGet()
        {
            Contactos = _svc.ObtenerTodos();
            Mensaje = TempData["Mensaje"] as string;
        }
    }
}
```

```
// Pages/Contactos/Index.cshtml
@page
@model IndexModel

<h1>Mis contactos</h1>

@if (!string.IsNullOrEmpty(Model.Mensaje))
{
    <div class="alert alert-success">
        @Model.Mensaje
    </div>
}

<a asp-page="Crear" class="btn btn-primary">
    + Nuevo contacto
</a>

@if (Model.Contactos.Count == 0)
{
    <p>No tienes contactos todavía.</p>
}
else
{
    <table class="table">
        @foreach (var c in Model.Contactos)
        {
            <tr>
                <td>@c.Nombre</td>
                <td>@c.Telefono</td>
                <td>@c.Email</td>
                <td>
                    <a asp-page="Editor">
                        Editar
                    </a>
                </td>
            </tr>
        }
    </table>
}
```



## Página Crear: formulario nuevo

```
// Pages/Contactos/Crear.cshtml.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using MiAgenda.Models;
using MiAgenda.Services;

namespace MiAgenda.Pages.Contactos
{
    public class CrearModel : PageModel
    {
        private readonly AgendaService _svc;

        public CrearModel(AgendaService svc)
        {
            _svc = svc;
        }

        [BindProperty]
        public Contacto Contacto { get; set; }

        public void OnGet() { }

        public IActionResult OnPost()
        {
            if (!ModelState.IsValid)
            {
                return Page();
            }

            _svc.Añadir(Contacto);
        }
    }
}
```

```
// Pages/Contactos/Crear.cshtml
@page
@model CrearModel

<h1>Nuevo contacto</h1>

<form method="post">
    <div>
        <label asp-for="Contacto.Nombre"></label>
        <input asp-for="Contacto.Nombre" />
        <span asp-validation-for="Contacto.Nombre">
        </span>
    </div>

    <div>
        <label asp-for="Contacto.Telefono"></label>
        <input asp-for="Contacto.Telefono" />
        <span asp-validation-for="Contacto.Telefono">
        </span>
    </div>

    <div>
        <label asp-for="Contacto.Email"></label>
        <input asp-for="Contacto.Email" />
        <span asp-validation-for="Contacto.Email">
        </span>
    </div>

    <button type="submit">Guardar</button>
    <a asp-page="Index">Cancelar</a>
</form>
```

✨ Los Tag Helpers `asp-for` generan labels, inputs y validación automáticamente. Si el POST falla, `return Page()` vuelve a mostrar el form con los valores introducidos y los mensajes de error.





## Página Editar: cargar y guardar

```
// Pages/Contactos/Editar.cshtml.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using MiAgenda.Models;
using MiAgenda.Services;

namespace MiAgenda.Pages.Contactos
{
    public class EditarModel : PageModel
    {
        private readonly AgendaService _svc;

        public EditarModel(AgendaService svc)
        {
            _svc = svc;
        }

        [BindProperty]
        public Contacto Contacto { get; set; }

        // GET /Contactos/Editar/5
        public IActionResult OnGet(int id)
        {
            Contacto = _svc.ObtenerPorId(id);

            if (Contacto == null)
            {
                return NotFound();
            }

            return Page();
        }
    }
}
```

```
// Pages/Contactos/Editar.cshtml
@page "{id:int}"
@model EditarModel

<h1>Editar contacto</h1>

<form method="post">
    @* Campo oculto para preservar el Id *@
    <input type="hidden" asp-for="Contacto.Id" />

    <div>
        <label asp-for="Contacto.Nombre"></label>
        <input asp-for="Contacto.Nombre" />
        <span asp-validation-for="Contacto.Nombre">
            </span>
        </div>

    <div>
        <label asp-for="Contacto.Telefono"></label>
        <input asp-for="Contacto.Telefono" />
        <span asp-validation-for="Contacto.Telefono">
            </span>
        </div>

    <div>
        <label asp-for="Contacto.Email"></label>
        <input asp-for="Contacto.Email" />
        <span asp-validation-for="Contacto.Email">
            </span>
        </div>

    <button type="submit">Guardar cambios</button>
    <a asp-page="Index">Cancelar</a>
</form>
```

🔗 @page "{id:int}" declara que la URL recibe un parámetro id entero. /Contactos/Editar/5 lo asigna al parámetro id de OnGet.



# Página Eliminar: confirmar y borrar

Patrón recomendado: GET muestra confirmación, POST ejecuta el borrado

```
// Pages/Contactos/Eliminar.cshtml.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using MiAgenda.Models;
using MiAgenda.Services;

namespace MiAgenda.Pages.Contactos
{
    public class EliminarModel : PageModel
    {
        private readonly AgendaService _svc;

        public EliminarModel(AgendaService svc)
        {
            _svc = svc;
        }

        [BindProperty]
        public Contacto Contacto { get; set; }

        // GET muestra la confirmación
        public IActionResult OnGet(int id)
        {
            Contacto = _svc.ObtenerPorId(id);
            if (Contacto == null) return NotFound();
            return Page();
        }
    }
}
```

```
// Pages/Contactos/Eliminar.cshtml
@page "{id:int}"
@model EliminarModel

<h1>Eliminar contacto</h1>

<div class="alert alert-danger">
    <p>¿Seguro que quieres eliminar a
        <strong>@Model.Contacto.Nombre</strong>?</p>
    <p>Teléfono: @Model.Contacto.Telefono</p>
</div>

<form method="post">
    <input type="hidden" asp-for="Contacto.Id" />
    <button type="submit" class="btn-danger">
        Sí, eliminar
    </button>
    <a asp-page="Index" class="btn">
        Cancelar
    </a>
</form>
```



Separar **GET (confirmar)** de **POST (borrar)** evita que un crawler o un enlace accidental borre datos. El borrado real solo ocurre al enviar el formulario.



# Validación de formularios

Anotaciones de datos + ModelState = validación automática en servidor y cliente

```
// Modelo + PageModel
// En el modelo
public class Contacto
{
    [Required(ErrorMessage = "El nombre es obligatorio")]
    [StringLength(50, MinimumLength = 2,
        ErrorMessage = "Entre 2 y 50 caracteres")]
    [Display(Name = "Nombre completo")]
    public string Nombre { get; set; }

    [Required(ErrorMessage = "El teléfono es obligatorio")]
    [Phone(ErrorMessage = "Teléfono no válido")]
    [RegularExpression(@"^[0-9+\s\-\]{9,15}$",
        ErrorMessage = "Formato: 600123456 o +34 600 123 456")]
    public string Telefono { get; set; }
}

// En el PageModel
public IActionResult OnPost()
{
    if (!ModelState.IsValid)
    {
        // ModelState contiene todos los errores
        // detectados por las anotaciones
        return Page();
    }
    // ... guardar
}
```



## Doble validación

ASP.NET valida en servidor (siempre) y genera scripts jQuery para validar en cliente (si el navegador tiene JS activo).



## ModelState

Objeto que recolecta todos los errores de validación. `ModelState.IsValid` es true solo si no hay ninguno. `ModelState["Nombre"].Errors` devuelve la lista de mensajes.



## Repopula el form

Al devolver `Page()`, los Tag Helpers `asp-for` vuelven a renderizar el form CON los valores introducidos y los mensajes de error debajo de cada campo. El usuario no tiene que reescribir todo.



Servidor



Cliente (jQuery)



ModelState.IsValid

# 06 / 06

---

## Lista de tareas con base de datos

Da el salto a persistencia real: Entity Framework Core con SQLite para que las tareas no se borren al reiniciar.

 EF Core  SQLite  Migraciones  CRUD persistente



# ¿Qué es Entity Framework Core?

” **EF Core** es el ORM (Object-Relational Mapper) oficial de .NET. Te permite leer y escribir en la base de datos usando objetos C# — sin escribir SQL manualmente. Define tus clases, configura la conexión, y EF Core se encarga de traducir tus operaciones a consultas SQL optimizadas.



## Modelos = tablas

Cada clase C# (Contacto, Tarea) se convierte en una tabla. Cada propiedad se convierte en una columna. Las relaciones entre clases se traducen en claves foráneas.



## LINQ to SQL

Escribes consultas con LINQ:

```
_db.Contactos.Where(c =>
c.Nombre.Contains("Ana")).ToList() . EF Core las
traduce a SQL: SELECT * FROM Contactos WHERE
Nombre LIKE %Ana% .
```



## Migraciones

Cuando cambias un modelo, generas una migración que actualiza el esquema de la BD. Igual que Git, pero para bases de datos. Sin perder datos existentes.

## PASO 1

# Instalar EF Core + SQLite

Tres paquetes NuGet y una cadena de conexión — eso es todo



```
# CLI — desde la carpeta del proyecto
# Desde la carpeta del proyecto
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet add package Microsoft.EntityFrameworkCore.Design

# Verificar instalación
dotnet ef --version
```

```
// appsettings.json
{
  "ConnectionStrings": {
    "Default": "Data Source=agenda.db"
  }
}
```



### Paquete Sqlite

Incluye el proveedor de SQLite para EF Core. No necesitas instalar SQLite por separado: el paquete NuGet trae todo lo necesario.



### Paquete Tools

Habilita los comandos `dotnet ef` (migraciones, scaffolding). Necesario solo en desarrollo, no en producción.



### Cadena de conexión

`Data Source=agenda.db` crea un archivo SQLite en la raíz del proyecto. SQLite guarda todo en un solo archivo — perfecto para demos y apps pequeñas.

**TOTAL:** 3 paquetes NuGet + 1 cadena en appsettings.json = **Listo**



## Modelo Tarea y DbContext

```
// Models/Tarea.cs
using System.ComponentModel.DataAnnotations;

namespace MiAgenda.Models
{
    public class Tarea
    {
        public int Id { get; set; }

        [Required]
        [StringLength(100)]
        public string Titulo { get; set; }

        public string? Descripcion { get; set; }

        public bool Completada { get; set; }

        public DateTime FechaCreacion { get; set; }
        = DateTime.Now;

        public DateTime? FechaLimite { get; set; }
    }
}
```

```
// Program.cs
builder.Services.AddDbContext<AgendaDbContext>(
    options => options.UseSqlite(
        builder.Configuration
            .GetConnectionString("Default")));
```

```
// Data/AgendaDbContext.cs
using Microsoft.EntityFrameworkCore;
using MiAgenda.Models;

namespace MiAgenda.Data
{
    public class AgendaDbContext : DbContext
    {
        public AgendaDbContext(
            DbContextOptions<AgendaDbContext> options)
            : base(options) { }

        // DbSet por cada tabla
        public DbSet<Tarea> Tareas { get; set; }
        public DbSet<Contacto> Contactos { get; set; }

        protected override void OnModelCreating(
            modelBuilder modelBuilder)
        {
            // Configuración adicional
            modelBuilder.Entity<Tarea>()
                .Property(t => t.Titulo)
                .IsRequired();
        }
    }
}
```

🔧 Cada `DbSet<T>` expone una tabla. Para usarla: `_context.Tareas.Add(tarea); await _context.SaveChangesAsync();`



# Migraciones: crear la base de datos

Convierte tus clases C# en tablas reales con dos comandos

```
# CLI de EF Core
# 1. Crear la migración inicial
# (genera código C# que crea las tablas)
dotnet ef migrations add Inicial

# 2. Aplicar la migración a la BD
# (ejecuta el código y crea agenda.db)
dotnet ef database update

# 3. Ver migraciones aplicadas
dotnet ef migrations list

# 4. Borrar la BD completa (¡cuidado!)
dotnet ef database drop
```

1

## Definir los modelos

Crea tus clases C# (Tarea, Contacto) con las propiedades y anotaciones que quieras.

2

## Crear migración

`dotnet ef migrations add Inicial` genera dos archivos en `Migrations/`: uno con el código C# de la migración y otro con el modelo snapshot. EF compara el snapshot actual con el anterior para detectar cambios.

3

## Aplicar a la BD

`dotnet ef database update` ejecuta las migraciones pendientes. Crea el archivo `agenda.db` (SQLite) con todas las tablas y columnas.

4

## Iterar

Si añades una propiedad a un modelo, repite los pasos: `dotnet ef migrations add AnadirCampo`, `dotnet ef database update`. EF hace un ALTER TABLE sin perder datos.





# CRUD de tareas con EF Core

Operaciones async para no bloquear el servidor

```
// Pages/Tareas/Index.cshtml.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using Microsoft.EntityFrameworkCore;
using MiAgenda.Data;
using MiAgenda.Models;

namespace MiAgenda.Pages.Tareas
{
    public class IndexModel : PageModel
    {
        private readonly AgendaDbContext _db;

        public IndexModel(AgendaDbContext db) => _db = db;

        public List<Tarea> Tareas { get; set; }

        public async Task OnGetAsync()
        {
            Tareas = await _db.Tareas
                .OrderByDescending(t => t.FechaCreacion)
                .ToListAsync();
        }

        // POST: marcar como completada
        public async IActionResult OnPostCompletarAsync(int id)
        {
            var tarea = await _db.Tareas.FindAsync(id);
            if (tarea != null)
            {
                tarea.Completada = !tarea.Completada;
                await _db.SaveChangesAsync();
            }
            return RedirectToPage();
        }
    }
}
```

```
// Pages/Tareas/Index.cshtml
@page
@model IndexModel

<h1>Mis tareas</h1>

<form method="post" asp-page-handler="Completar">
    <input type="number" name="id" hidden />
</form>

@foreach (var t in Model.Tareas)
{
    <div class="@t.Completada ? "done" : """>
        <strong>@t.Titulo</strong>
        <small>@t.FechaCreacion.ToString("dd/MM")</small>

        <form method="post"
            asp-page-handler="Completar"
            asp-route-id="@t.Id">
            <button>✓</button>
        </form>

        <form method="post"
            asp-page-handler="Borrar"
            asp-route-id="@t.Id">
            <button>x</button>
        </form>
    </div>
}
```

⚡ OnPostCompletarAsync y OnPostBorrarAsync son handlers. `asp-page-handler="Completar"` ejecuta OnPostCompletarAsync. `asp-route-id` pasa el parámetro.



# ¡Enhorabuena!

Has llegado hasta aquí. Estos son los siguientes pasos para seguir aprendiendo.

## LO QUE HAS APRENDIDO

- ✓ Qué es ASP.NET Core y Razor Pages
- ✓ Sintaxis Razor: @, bloques, directivas, Tag Helpers
- ✓ PageModel: OnGet, OnPost, [BindProperty], async
- ✓ CRUD completo de contactos en memoria
- ✓ Entity Framework Core con SQLite y migraciones
- ✓ CRUD completo de tareas persistente

## → SIGUIENTES PASOS



### Autenticación de usuarios

ASP.NET Core Identity: registro, login, roles. Permite que cada usuario tenga sus propias tareas y contactos.



### Despliegue en producción

Publica tu app en Azure App Service, Render o un contenedor Docker. `dotnet publish -c Release` genera los archivos listos.



### Razor Components

Da el salto a Blazor Server para añadir interactividad rica sin escribir JavaScript. Combina con Razor Pages en la misma app.



## Recursos recomendados:

[learn.microsoft.com/aspnet/core](https://learn.microsoft.com/aspnet/core) — Documentación oficial

[github.com/dotnet/AspNetCore.Docs](https://github.com/dotnet/AspNetCore.Docs) — Ejemplos en GitHub

[dotnet.microsoft.com/learn](https://dotnet.microsoft.com/learn) — Tutoriales gratuitos

¡Feliz programación con C# y Razor Pages! ❤️